# TasksManager — Central Project Documentation

> **Document Version**: 1.0.0
> **Last Codebase Audit**: October 2026
> **Repository Type**: Full-Stack Monorepo ( `client/` + `server/` )
> **Source of Truth**: Current source code in `d:\CodingProjectsByMe\TasksManager`

---

## 1. System Overview & Purpose

**TasksManager** is a full-stack daily timesheet, task management, and communication follow-up platform designed for professionals, engineering departments, and workflow coordinators.

The application solves key operational challenges:

1. **Granular Calendar Tracking**: Categorizes and filters work across macro and micro time horizons (Year → Month → 6-Day Work Week [Mon–Sat] → Day) without overwhelming users.
2. **Multilingual AI Task Extraction**: Parses free-form, informal, and conversational work notes written in English, Marathi, Roman Marathi, Hindi, or Hinglish into structured, professional task entries.
3. **Structured Follow-Up Communication**: Dedicated interaction logs (calls, WhatsApp, emails, meetings) with attachment proofs and scheduled next contact dates.
4. **Automated Managerial Escalations**: Automatically flags overdue items and dispatches HTML email escalation notifications with complete audit trails to Managers, Heads of Department (HOD), and Deputy HODs when communication thresholds are exceeded.
5. **Secure Public Read-Only Portals**: Tokenized, live-updating status portals for external viewers or executive review without granting write access to the administrative workspace.
6. **Multi-Format Audit Reporting**: In-memory generation of landscape PDF registers, multi-sheet Excel workbooks, and structured ZIP archives containing consolidated documents and media attachments.

---

## 2. Implementation Status Summary

To ensure accuracy, all features and subsystems documented below are categorized by their verified state in the codebase:

| Subsystem / Capability | Status | Evidence in Codebase |
|---|---|---|
| Task CRUD & Filtering | **Implemented** | `server/src/controllers/taskController.ts`, `client/components/tasks/TaskTable.tsx` |
| Two-Phase Task Resequencing | **Implemented** | `server/src/controllers/taskController.ts` (`resequenceTaskIds`) |
| Multilingual AI Parsing | **Implemented** | `server/src/services/aiService.ts`, `server/src/controllers/aiDraftController.ts` |
| AI Multi-Provider Cascade | **Implemented** | Gemini interactions/generateContent, Groq, xAI Grok, BYOK headers |
| AI Server State Persistence | **Implemented** | `server/src/models/AiDraftState.ts`, `server/src/routes/aiDraft.ts` |
| Follow-Ups Communication Log | **Implemented** | `server/src/models/FollowUp.ts`, `client/components/follow-ups/FollowUpWorkspace.tsx` |
| Multi-Channel Attachments | **Implemented** | Dual: Cloudinary (`uploadService.ts`) & MongoDB Buffer (`attachmentController.ts`) |
| Automated Email Escalations | **Implemented** | `server/src/services/emailService.ts`, `server/src/controllers/escalationController.ts` |
| PDF, Excel, ZIP Export Engine | **Implemented** | `server/src/controllers/exportController.ts` (`pdfkit`, `exceljs`, `archiver`) |
| Public Status Portal (`/status/:token`) | **Implemented** | `client/app/status/[token]/page.tsx`, `server/src/controllers/publicController.ts` |
| Master Passcode & Auto-Lock | **Implemented** | `client/lib/sessionManager.ts`, `client/components/auth/PasswordGate.tsx` |
| Cross-Tab Session Sync | **Implemented** | `BroadcastChannel` + `localStorage` storage events |
| Standard `givenBy` Normalization | **Implemented** | `server/src/utils/normalizeGivenBy.ts`, `client/lib/normalizeGivenBy.ts` |
| Dynamic 4-Theme Design System | **Implemented** | `client/app/globals.css`, `client/app/settings/page.tsx` (Light, Dark, Slate, Sketch) |
| Model Context Protocol (MCP) Suite | **Implemented** | Filesystem, Git, Serena, Context7 configured in `.agents/mcp_config.json` & verified via `npm run mcp:verify` |
| Multi-User RBAC / Auth0 / NextAuth | **Not Implemented** | Monolithic Master Passcode gate used; no individual user accounts or JWTs. |
| Background Cron Scheduler | **Partially Implemented** | Polling intervals on client (5s/15s) trigger health wake-ups; no standalone cron daemon. |

# 3. Users and Primary Use Cases

## Main User Personas

1. **Primary Operator / Staff Engineer**:
   - Enters daily timesheets manually or pastes free-form mixed-language notes into the AI Assistant.
   - Logs phone, WhatsApp, email, or meeting follow-ups against pending deliverables.
   - Attaches audio clips, call recordings, PDFs, or screenshots as proof of communication.
2. **Department Manager / HOD / DyHOD**:
   - Receives automated escalation emails when follow-up attempts hit configured thresholds.
   - Reviews overdue tasks, delayed dependencies, and reason/remark histories.
3. **External Stakeholders / Clients / Auditors**:
   - Accesses the read-only `/status/[token]` portal to monitor live project progress.
   - Downloads official PDF reports, Excel timesheets, or calendar-structured ZIP bundles.

# 4. Application Architecture

## High-Level Architecture Diagram



# 5. Main Pages and Navigation Routes

### Client Routes ( `client/app` )

| Route Path | Access Control | Purpose | Key Components |
|---|---|---|---|
| `/` | Protected (Passcode) | Main Dashboard: aggregate metrics, current day/month scope, quick filters. | `SummaryCards` , `DashboardScopeBar` , `TaskTable` |
| `/tasks` | Protected (Passcode) | Full task register with pagination, search, sorting, and inline editing. | `TaskScopeNav` , `TaskTable` , `TaskRow` , `FilterSortPanel` |
| `/follow-ups` | Protected (Passcode) | Dedicated communication log interface for tasks with active outreach. | `FollowUpWorkspace` , `FollowUpPanel` , `FollowUpTimeline` |
| `/settings` | Protected (Passcode) | Configuration for UI themes, escalation rules, notifications, and security. | `EscalationRecipientsManager` , Theme Selector, Passcode Form |
| `/help` | Protected (Passcode) | Searchable FAQ hub with category browsing and keyboard shortcuts. | Category cards, accordion FAQ items, search bar |
| `/status/[token]` | **Public** (Tokenized) | Read-only external status portal with live task status feed and client search. | `PublicStatusPage` , `TaskCardSkeleton` , Category Pills |

# 6. Detailed Backend API Specifications

Base URL: `http://localhost:4000/api` (Local) or `/api` (via Next.js rewrite in production).

### 6.1 Task Endpoints ( `/api/tasks` )

- `GET /api/tasks` : Query tasks with filtering ( `search` , `status` , `priority` , `givenBy` , `year` , `month` , `weekStart` , `day` , `hasFollowUps` , `overdueOnly` , `escalatedOnly` , `sort` , `order` , `page` , `limit` ).
- `GET /api/tasks/given-by` : Returns deduplicated, sorted list of standard and existing `givenBy` values.
- `POST /api/tasks` : Create a new task. Generates atomic `TK-XXXX` taskId. Automatically syncs reason/remarks fields based on work status.
- `PUT /api/tasks/:id` : Update task fields and synchronizes status reasons.
- `DELETE /api/tasks/:id` : Soft-deletes task ( `isDeleted: true` ), marks taskId with `DEL-` prefix, cascades soft-delete to follow-ups, and triggers two-phase resequencing.
- `DELETE /api/tasks/bulk` : Bulk soft-deletes up to 500 tasks with batch resequencing.
- `POST /api/tasks/:id/restore` : Restores soft-deleted task, restores its cascade-deleted follow-ups, and resequences IDs.
- `POST /api/tasks/:id/escalate` : Manually triggers escalation email dispatch for a specific task.

### 6.2 Follow-Up Endpoints ( `/api/tasks/:taskId/follow-ups` )

- `GET /` : Lists all active follow-ups for a task, enriched with attachments, sorted newest-first.
- `POST /` : Creates a follow-up. Enforces duplicate detection (within ±60s with identical communication text). Increments `followUpNumber` .
- `GET /:id` : Retrieves single follow-up details.
- `PUT /:id` : Updates follow-up communication or scheduled dates.
- `DELETE /:id` : Soft-deletes follow-up with mandatory `reason` .
- `POST /:id/attachments` : Multer upload of up to 5 files to Cloudinary storage.
- `DELETE /:id/attachments/:attachmentId` : Removes attachment from Cloudinary and model.

### 6.3 Local/MongoDB Attachment Endpoints

- `GET /api/tasks/:taskId/follow-ups/:followUpId/attachments` : Lists stored binary attachments.
- `POST /api/tasks/:taskId/follow-ups/:followUpId/attachments` : Uploads files into MongoDB binary Buffer (up to 100MB; whitelisted images, PDFs, office docs, audio, and video).
- `GET /api/f/:id/:filename?` : Downloads or inline-streams attachment data with proper MIME headers.
- `DELETE /api/tasks/:taskId/follow-ups/:followUpId/attachments/:id` : Deletes attachment record from MongoDB.

### 6.4 AI Assistant Endpoints ( `/api/ai-draft` )

- `GET /api/ai-draft/providers` : Returns status of configured AI providers (Gemini, Groq, xAI Grok).
- `GET /api/ai-draft/state` : Fetches persistent in-progress draft state for user session.
- `PUT /api/ai-draft/state` : Persists raw text, active step, and uncommitted draft cards to MongoDB.
- `DELETE /api/ai-draft/state` : Clears saved AI draft state.
- `POST /api/ai-draft` : Processes free-form multilingual text and returns an array of structured task drafts.
- `POST /api/ai-draft/regenerate` : Refines a single task card based on targeted natural-language instructions.

### 6.5 Export Endpoints ( `/api/export` )

- `GET /api/export/pdf` : Generates landscape A4 PDF report with color-coded status badges and follow-up audit cards.
- `GET /api/export/excel` : Generates styled Excel spreadsheet with Tasks Register, Follow-ups Register, and Summary statistics.
- `GET /api/export/zip` : Generates complete hierarchical ZIP bundle containing consolidated PDFs/Excel files and raw attachment proofs.

### 6.6 Escalation & Settings Endpoints

- `GET /api/escalations` : Retrieves escalation threshold and TO/CC/BCC recipient lists.
- `PUT /api/escalations` : Updates escalation thresholds and recipient configurations.
- `GET /api/settings` : Returns public sharing status and share token (omits master password hash).
- `PATCH /api/settings` : Toggles public link sharing status.
- `POST /api/settings/regenerate-token` : Regenerates 16-byte random hex share token.

- `POST /api/settings/verify-password`: Compares client passcode against salted SHA256 hash.
- `POST /api/settings/update-password`: Sets new master passcode in MongoDB.
- `GET /api/public/status/:token`: Public endpoint returning categorized active tasks for valid share tokens.
- `GET /api/summary`: Returns status counters, overdue follow-up counts, and escalated task counts.

---

## 7. Database Models and Data Schemas

The database utilizes MongoDB via Mongoose 8.5.

### 7.1 `Task` Schema (`server/src/models/Task.ts`)

```
{
  userId: string | null,           // Tenant identifier (default: null)
  taskId: string,                  // Unique sequential key: "TK-0001", "TK-0002"
  title: string,                   // Task title (required, max 300 chars, trimmed)
  description: string,             // Full task details / bullet subtasks
  givenBy: string,                 // Assigned by (normalized on write)
  contactPerson: string,           // Outreach contact name
  priority: 'High' | 'Medium' | 'Low', // Default: 'Medium'
  workStatus: 'InProgress' | 'Pending' | 'Completed', // Default: 'Pending'
  reason: string,                  // Primary status reason (synchronized)
  remarks: string,                 // Primary completion remarks
  inProgressReason: string,        // Preserved InProgress notes
  pendingReason: string,           // Preserved Pending notes
  completedRemarks: string,        // Preserved Completed remarks
  date: Date,                      // Scheduled task date (required)
  dueDate: Date | null,            // Deadline date (optional)
  isDeleted: boolean,              // Soft delete flag (default: false, indexed)
  deletedAt: Date | null,          // Deletion timestamp
  deletedReason: string,           // Reason for deletion
  createdAt: Date,                 // Auto-managed timestamp
  updatedAt: Date                  // Auto-managed timestamp
}
```

**Indexes**: `{ taskId: 1 }` (unique), `{ date: -1, workStatus: 1 }`, `{ workStatus: 1 }`, `{ priority: 1 }`, `{ isDeleted: 1 }`, Text index on `{ title, description, givenBy }`.

### 7.2 `FollowUp` Schema ( `server/src/models/FollowUp.ts` )

```
{
  taskId: ObjectId,                // Reference to Task (required, indexed)
  followUpNumber: number,          // Sequential attempt number for parent task (1, 2, 3...)
  followUpDate: Date,              // Timestamp communication occurred (required)
  method: 'Phone' | 'WhatsApp' | 'Email' | 'InPerson' | 'Teams' | 'GoogleMeet' | 'Other',
  methodOther: string,             // Text when method === 'Other'
  contactPerson: string,           // Contact contacted
  communicated: string,            // What was stated/requested (max 5000 chars)
  responseReceived: string,        // Client/recipient reply (max 5000 chars)
  notes: string,                   // Internal observations (max 2000 chars)
  nextAction: string,              // Pending deliverable (max 2000 chars)
  nextFollowUpDate: Date | null,   // Scheduled next contact timestamp (indexed)
  isDeleted: boolean,              // Soft delete flag
  deletedReason: string,           // Reason for removal
  attachments: [                   // Cloudinary attachments array
    { url: string, public_id: string, filename: string }
  ]
}
```

### 7.3 `FollowUpAttachment` Schema ( `server/src/models/FollowUpAttachment.ts` )

Stores binary attachment proofs directly in MongoDB as Buffers or falls back to server disk storage:

```
{
  followUpId: ObjectId,            // Reference to FollowUp (indexed)
  taskId: ObjectId,                // Reference to Task (indexed)
  originalName: string,            // Client file name
  storedName?: string,             // File system safe name (legacy)
  mimeType: string,                // MIME type
  sizeBytes: number,               // File size in bytes
  data: Buffer                     // Binary file content
}
```

### 7.4 `EscalationSettings` Schema ( `server/src/models/EscalationSettings.ts` )

```
{
  threshold: number,               // Number of follow-ups before escalation (default: 3)
  enabled: boolean,                // System active toggle (default: false)
  toRecipients: [                  // Primary recipients (Manager, HOD, DyHOD)
    { email: string, tag: string }
  ],
  ccRecipients: [                  // CC recipients
    { email: string, tag: string }
  ],
  bccRecipients: [                 // BCC recipients
    { email: string, tag: string }
  ],
  managerEmail: string,            // Legacy backward-compatibility fallback
  hodEmail: string,
  dyhodEmail: string,
  ccEmail: string
}
```

### 7.5 `Settings` Schema ( `server/src/models/Settings.ts` )

```
{
  isPublicShareEnabled: boolean,   // Toggles /status/[token] availability
  publicShareToken: string,        // 32-character random hex token
  masterPasswordHash: string|null, // SHA256(salt + passcode)
  passwordUpdatedAt: Date | null
}
```

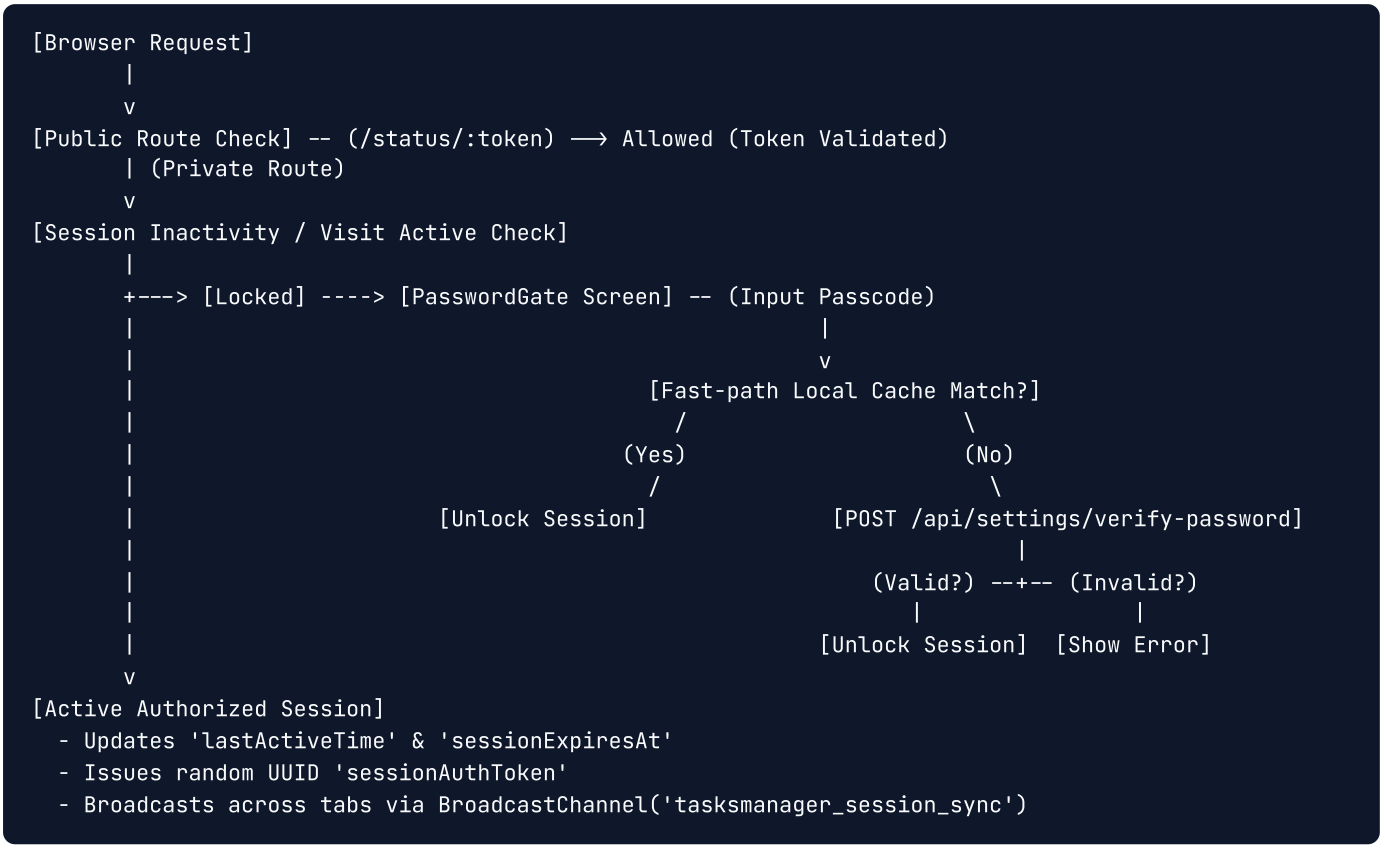
### 7.6 `AiDraftState` Schema ( `server/src/models/AiDraftState.ts` )

```
{
  userId: string,                  // Session/tenant key (default: 'default')
  rawText: string,                 // Uncommitted prompt input
  step: 'input' | 'preview',       // Active modal view
  drafts: Array<any>,              // Parsed candidate task cards
  clearedAt: Date | null
}
```

---

## 8. Authentication and Security Model

The application employs a defense-in-depth single-tenant security gate designed for private workstation operations:




1. **Passcode Protection**:
   - Master passcode is hashed with a server-side salt using SHA-256 ( `tasksmanager_salt_<passcode>` ).
   - Defaults to `Sun123@` or `Sushant2026@` if uninitialized in MongoDB.

- Updated in `Settings → Security` via `POST /api/settings/update-password`.

2. **Inactivity Auto-Lock**:
   - Configurable timeout options: 5 min, 10 min, 15 min (default), 30 min, 1 hr, or Never.
   - User activity events (`mousemove`, `keydown`, `wheel`, `scroll`, `touchstart`) throttled to refresh timestamp every 4 seconds.
   - Background tab heartbeat checks session expiry every 15 seconds.
3. **Cross-Tab Synchronization**:
   - Utilizes `BroadcastChannel('tasksmanager_session_sync')` alongside window `storage` listeners.
   - Unlocking or locking in one tab instantly updates all open tabs.
4. **Tokenized Public Route Isolation**:
   - The `/status/:token` route completely bypasses `PasswordGate`.
   - Backend checks `isPublicShareEnabled === true` and strictly matches the 32-character token before returning sanitized, active tasks.

---

## 9. Key Business Rules & Technical Invariants

### 9.1 Two-Phase Task ID Resequencing

To prevent gaps in task numbers when tasks are deleted or restored, the system enforces sequential IDs (`TK-0001`, `TK-0002`...):

1. Any soft-deleted task has its `taskId` updated to `DEL—<originalId>-<_id>`.
2. Active tasks are sorted ascending by `createdAt`.
3. In MongoDB, updating unique indexed keys sequentially causes collision errors (`E11000 duplicate key`). The system executes a **Two-Phase Bulk Write**:
   - **Phase 1**: Assigns temporary unique IDs (`TEMP—<id>`).
   - **Phase 2**: Reassigns clean sequential IDs (`TK-0001`, `TK-0002`).
4. The atomic `Counter` sequence is synchronized with active task count.

### 9.2 Status and Reason Synchronization

Tasks maintain separate storage for notes across all three lifecycles:

- `workStatus === 'InProgress'`: reads/writes `inProgressReason` and mirrors to `reason`.
- `workStatus === 'Pending'`: reads/writes `pendingReason` and mirrors to `reason`.
- `workStatus === 'Completed'`: reads/writes `completedRemarks` and mirrors to `remarks`.
- Switching status does not overwrite notes from previous stages; users can review complete stage notes under `Status History & Notes`.

### 9.3 6-Day Work Week Indexing

In `buildQuery.ts` and `dates.ts`, weeks are defined from **Monday to Saturday (6 days)**:

```
const weekEnd = new Date(weekStartDate);
weekEnd.setUTCDate(weekEnd.getUTCDate() + 5);
weekEnd.setUTCHours(23, 59, 59, 999);
```

Sunday is treated as a rest boundary and excluded from the active week range.

### 9.4 Name Normalization ( `givenBy` )

In `normalizeGivenBy.ts` :

- Standard names dictionary prioritizes known faculty/staff (e.g., "Sachin" defaults to "Sachin Oak Sir").
- Strips honorifics ( `sir` , `ma'am` , `mam` , `dr` , `prof` ) during matching and formats them properly.
- Splits multi-person assignments using `&` , `and` , or `,` and normalizes each entity individually.

---

## 10. AI Engine Integration Architecture

The AI module ( `server/src/services/aiService.ts` ) implements an autonomous multi-tier fallback cascade:

```
[Raw Multilingual Text (Marathi / Hinglish / English)]
                        |
                        v
        +----------------------------------+
        |   1. Custom BYOK API Key?        | (Gemini / Groq / Grok)
        +----------------------------------+
                        | (Fails or None)
                        v
        +----------------------------------+
        |   2. Server Primary Gemini       | (gemini-3.8-flash)
        +----------------------------------+
                        | (Quota/Rate Limit/Timeout)
                        v
        +----------------------------------+
        |   3. Server Gemini Lite Fallback | (gemini-3.5-flash-lite)
        +----------------------------------+
                        | (Separate Quota Pool Fails)
                        v
        +----------------------------------+
        |   4. Backup Gemini Server Keys   | (GEMINI_API_KEY_2, 3...)
        +----------------------------------+
                        | (All Gemini Exhausted)
                        v
        +----------------------------------+
        |   5. Server Groq Ultra-Fast API  | (openai/gpt-oss-120b or qwen3.8)
        +----------------------------------+
                        | (Fails or Not Configured)
                        v
        +----------------------------------+
        |   6. Server xAI Grok API         | (grok-2-latest)
        +----------------------------------+
                        | (All Exhausted)
                        v
        [Rate Limit / Quota Exceeded Modal with Countdown]
```

- **Output Guarantee**: Strips markdown code fences ( `json...` ) and validates with `JSON.parse` .
- **Sub-task Handling Rule**: Sub-tasks ( `a, b, c` , indented dots) are merged into the main task description as explicit bullet lines ( `\n-` ) rather than creating fragmented task records.
- **Auto-Fill Rules**: Words like *"baaki aahe"*, *"pending"*, *"rahilay"* map to `Pending` ; *"chalu aahe"*, *"karat aahe"* map to `InProgress` ; *"kela"*, *"jhala"*, *"done"* map to `Completed` .

---

## 11. External Services and Dependencies

| Dependency | Purpose | Configuration / Secrets |
|---|---|---|
| **MongoDB Atlas / Local** | Primary document database | `MONGODB_URI` in `server/.env` |
| **Google GenAI SDK** | Primary AI task parsing engine | `GEMINI_API_KEY` (with backup keys `2` , `3` ) |
| **Groq API** | High-throughput AI fallback | `GROQ_API_KEY` , `GROQ_MODEL` |
| **xAI Grok API** | Tertiary AI fallback | `XAI_API_KEY` , `XAI_MODEL` |
| **Cloudinary** | Image, video, and PDF attachment hosting | `CLOUDINARY_CLOUD_NAME` , `CLOUDINARY_API_KEY` , `CLOUDINARY_API_SECRET` |
| **Nodemailer / SMTP** | Automated escalation email delivery | `SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASS` , `FROM_EMAIL` |
| **Playwright Chromium** | Headless browser testing & PDF compilation | `@playwright/test` in `client/package.json` |
| **ExcelJS & PDFKit** | In-memory spreadsheet & PDF report generator | Installed in `server/package.json` |
| **Archiver** | Streaming ZIP archive generation | Installed in `server/package.json` |

### 11.1 Model Context Protocol (MCP) Server Suite

The repository is configured with 4 specialized Model Context Protocol (MCP) servers to enable high-fidelity AI-assisted inspection, semantic navigation, and live framework reference:

| MCP Server | Package / Command | Scope & Capabilities |
|---|---|---|
| **Filesystem MCP** | `@modelcontextprotocol/server-filesystem` | Direct filesystem traversal, file tree inspection, read/write within workspace root. |
| **Git MCP** | `mcp-server-git` (via `uvx` ) | Git history analysis, branch inspection, commit diffs, and safe refactoring checkpoints. |
| **Serena MCP** | `oraios/serena` (via `uvx` ) | Semantic code intelligence, AST symbol tracing, reference lookup, and large-scale refactoring. |
| **Context7 MCP** | `@upstash/context7-mcp` | Up-to-date documentation retrieval for Next.js, React, TypeScript, Tailwind, and libraries. |

**Configuration Locations:**

- Workspace / Project: `.agents/mcp_config.json`
- IDE Workspace Compatibility: `.cursor/mcp.json`
- User Global Environment: `C:\Users\gupta\.gemini\config\mcp_config.json`

**Verification Command:**

```
npm run mcp:verify
```

---

## 12. Environment Configuration Reference

### Server ( `server/.env` )

```
# Database
MONGODB_URI=mongodb://localhost:27017/taskmanager

# Networking
PORT=4000
CLIENT_URL=http://localhost:3000
TIMEZONE=Asia/Kolkata

# AI Engines (Server Keys)
GEMINI_API_KEY=your_gemini_key_here
GEMINI_API_KEY_2=your_gemini_backup_key_here
GROQ_API_KEY=your_groq_key_here
GROQ_MODEL=openai/gpt-oss-120b
XAI_API_KEY=your_xai_key_here
XAI_MODEL=grok-2-latest

# Cloudinary Storage
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_api_secret

# Escalation SMTP Mail Server
SMTP_HOST=smtp.ethereal.email
SMTP_PORT=587
SMTP_USER=your_smtp_username
SMTP_PASS=your_smtp_password
FROM_EMAIL=no-reply@tasksmanager.com
```

### Client ( `client/.env.local` )

```
# Local development backend pointer
NEXT_PUBLIC_API_URL=http://localhost:4000
BACKEND_API_URL=http://localhost:4000
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

---

## 13. Current Limitations and Known Gotchas

1. **Render Free-Tier Cold Starts**:
   - Free instances spin down after inactivity.
   - *Mitigation in code*: `TaskContext.tsx` issues a silent warm-up ping to `/api/health` upon client initialization.
2. **Port 4000 Conflicts**:
   - Zombie Node processes can lock port 4000 on Windows.
   - *Mitigation in code*: Added `npm run clean:ports` ( `npx kill-port 3000 4000` ) in root `package.json`.
3. **Dual Attachment Storage Models**:
   - Follow-up attachments uploaded via `/api/tasks/:id/follow-ups/:id/attachments` go to **Cloudinary**.
   - Attachments uploaded via `/api/tasks/:id/follow-ups/:id/attachments` route in `attachments.ts` store binary buffers in **MongoDB** ( `FollowUpAttachment` ).
   - The export engine seamlessly handles both, fetching Cloudinary URLs via HTTP and reading MongoDB buffers directly.
4. **Single-Tenant Workspace**:
   - The application does not maintain distinct user tables ( `userId` defaults to `null` or `'default'` ). A single master passcode protects all administrative views.

---

## 14. Permanent Update Protocol

When requested to **"Update my project documentation"**, follow this deterministic procedure:

1. Re-analyze the codebase ( `git status` , file diffs, package updates, new routes, model changes).
2. Compare the current state against this document ( `PROJECT_DOCUMENTATION.md` ).
3. Update all modified endpoints, models, business rules, and UI features.
4. Maintain accuracy markers (**Implemented**, **Partially Implemented**, **Inferred**, **Unknown**).
5. Compile and regenerate `PROJECT_DOCUMENTATION.pdf` using the headless Chromium script to mirror this file.
6. Present a concise changelog summary of detected differences.